

For Loops in Python

Known Repetition, Collections and Nested Loops

Programming Fundamentals Team

SETU

2026-06-17

Outline

1. Learning outcomes

2. Why another loop?

3. For loop basics

4. Using range()

5. Accumulators with for loops

6. Iterating through strings

7. Iterating through lists

8. Nested loops

9. Pattern examples

10. Common mistakes

11. Bringing it together

12. Summary

Outline

1. Learning outcomes

2. Why another loop?

3. For loop basics

4. Using range()

5. Accumulators with for loops

6. Iterating through strings

7. Iterating through lists

8. Nested loops

9. Pattern examples

10. Common mistakes

11. Bringing it together

12. Summary

1.1 By the end of this lecture

You should be able to:

- Explain when a for loop is useful
- Use `range()` with start, stop and step values
- Iterate through strings
- Iterate through lists
- Use selection inside a loop
- Use accumulators with for loops
- Write nested loops
- Create simple text patterns

Outline

1. Learning outcomes

2. Why another loop?

3. For loop basics

4. Using range()

5. Accumulators with for loops

6. Iterating through strings

7. Iterating through lists

8. Nested loops

9. Pattern examples

10. Common mistakes

11. Bringing it together

12. Summary

2.1 Review: while loop

```
i = 1

while i <= 10:
    print(i)
    i += 1
```

Use `while` when we may not know how many repetitions are needed.

2.2 A simpler way when we know the count

```
for i in range(1, 11):  
    print(i)
```

A for loop is useful when we know what sequence we want to process.

2.3 While versus for

while loop

Use when repetition depends on a condition.

Examples:

- password validation
- menu loops
- sentinel loops

for loop

Use when processing a sequence.

Examples:

- numbers in a range
- characters in a string
- items in a list

Outline

1. Learning outcomes

2. Why another loop?

3. For loop basics

4. Using range()

5. Accumulators with for loops

6. Iterating through strings

7. Iterating through lists

8. Nested loops

9. Pattern examples

10. Common mistakes

11. Bringing it together

12. Summary

3.1 Syntax

```
for variable in sequence:  
    statements
```

Examples of sequences:

- `range(5)`
- a string such as `"Python"`
- a list such as `[70, 80, 90]`

3.2 First for loop

```
for i in range(5):  
    print(i)
```

Output:

```
0  
1  
2  
3  
4
```

3.3 Important: range stops before the end

```
range(5)
```

Produces:

0, 1, 2, 3, 4

It does not include 5.

Outline

1. Learning outcomes

2. Why another loop?

3. For loop basics

4. Using range()

5. Accumulators with for loops

6. Iterating through strings

7. Iterating through lists

8. Nested loops

9. Pattern examples

10. Common mistakes

11. Bringing it together

12. Summary

4.1 Start and stop

```
for i in range(1, 11):  
    print(i)
```

This prints 1 to 10.

`range(start, stop)` includes the start but excludes the stop.

4.2 Step value

```
for i in range(0, 21, 2):  
    print(i)
```

This prints even numbers from 0 to 20.

4.3 Counting backwards

```
for i in range(10, 0, -1):  
    print(i)
```

This prints 10 down to 1.

4.4 Activity: predict the output

```
for i in range(2, 12, 3):  
    print(i)
```

Questions:

- What is the first value?
- What is added each time?
- Why is 12 not printed?

4.5 Quick challenges

Write for loops to print:

1. 1 to 20
2. 10 to 1
3. Multiples of 5 from 5 to 50
4. Odd numbers from 1 to 19

Outline

1. Learning outcomes

2. Why another loop?

3. For loop basics

4. Using range()

5. Accumulators with for loops

6. Iterating through strings

7. Iterating through lists

8. Nested loops

9. Pattern examples

10. Common mistakes

11. Bringing it together

12. Summary

5.1 Sum numbers 1 to 100

```
total = 0

for i in range(1, 101):
    total += i

print(total)
```

5.2 Calculate an average

```
total = 0

for i in range(5):
    mark = int(input("Enter mark: "))
    total += mark

average = total / 5
print("Average:", average)
```

5.3 Add validation

```
total = 0

for i in range(5):
    mark = int(input("Enter mark: "))

    while mark < 0 or mark > 100:
        mark = int(input("Invalid. Try again: "))

    total += mark

print("Average:", total / 5)
```

This combines for, while, and or.

Outline

1. Learning outcomes

2. Why another loop?

3. For loop basics

4. Using range()

5. Accumulators with for loops

6. Iterating through strings

7. Iterating through lists

8. Nested loops

9. Pattern examples

10. Common mistakes

11. Bringing it together

12. Summary

6.1 Strings are sequences

```
for letter in "Python":  
    print(letter)
```

Output:

P
y
t
h
o
n

6.2 Count vowels

```
word = input("Enter word: ")
vowels = 0

for letter in word:
    if letter in "aeiouAEIOU":
        vowels += 1

print("Vowels:", vowels)
```

6.3 Activity: count spaces

Write a program that asks for a sentence and counts how many spaces it contains.

Hint:

```
if character == " ":
```

6.4 Solution: count spaces

```
sentence = input("Enter a sentence: ")

space_count = 0

for character in sentence:
    if character == " ":
        space_count += 1

print("Number of spaces:", space_count)
```

Outline

1. Learning outcomes

2. Why another loop?

3. For loop basics

4. Using range()

5. Accumulators with for loops

6. Iterating through strings

7. Iterating through lists

8. Nested loops

9. Pattern examples

10. Common mistakes

11. Bringing it together

12. Summary

7.1 What is a list?

A list stores multiple values in one variable.

```
marks = [75, 60, 80, 90]
```

A for loop is ideal for processing every item in a list.

7.2 Print each mark

```
marks = [75, 60, 80, 90]
```

```
for mark in marks:  
    print(mark)
```

7.3 Total and average

```
marks = [75, 60, 80, 90]  
total = 0
```

```
for mark in marks:  
    total += mark
```

```
average = total / len(marks)  
print("Average:", average)
```

7.4 Find the highest mark

```
marks = [75, 60, 80, 90]
highest = marks[0]

for mark in marks:
    if mark > highest:
        highest = mark

print("Highest:", highest)
```

7.5 Count passes

```
marks = [75, 40, 32, 81, 55]
passes = 0

for mark in marks:
    if mark >= 40:
        passes += 1

print("Passes:", passes)
```

Outline

1. Learning outcomes

2. Why another loop?

3. For loop basics

4. Using range()

5. Accumulators with for loops

6. Iterating through strings

7. Iterating through lists

8. Nested loops

9. Pattern examples

10. Common mistakes

11. Bringing it together

12. Summary

8.1 What is a nested loop?

A nested loop is a loop inside another loop.

Outer loop controls the main repetition.

Inner loop completes all of its repetitions for each one repetition of the outer loop.

8.2 Rows and Seats Analogy

Imagine a classroom:

- 3 rows
- 4 seats in each row

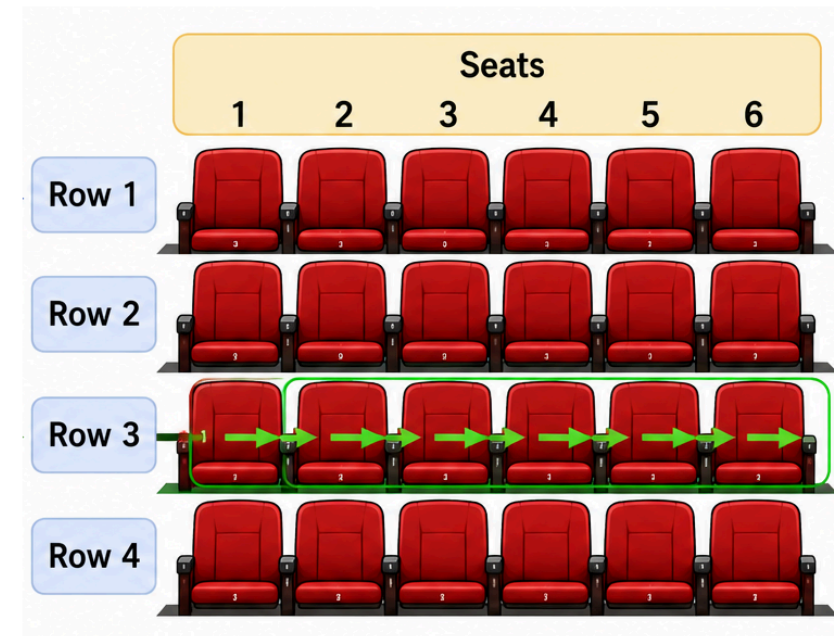
For every row, check every seat.

The **outer loop** selects the row.

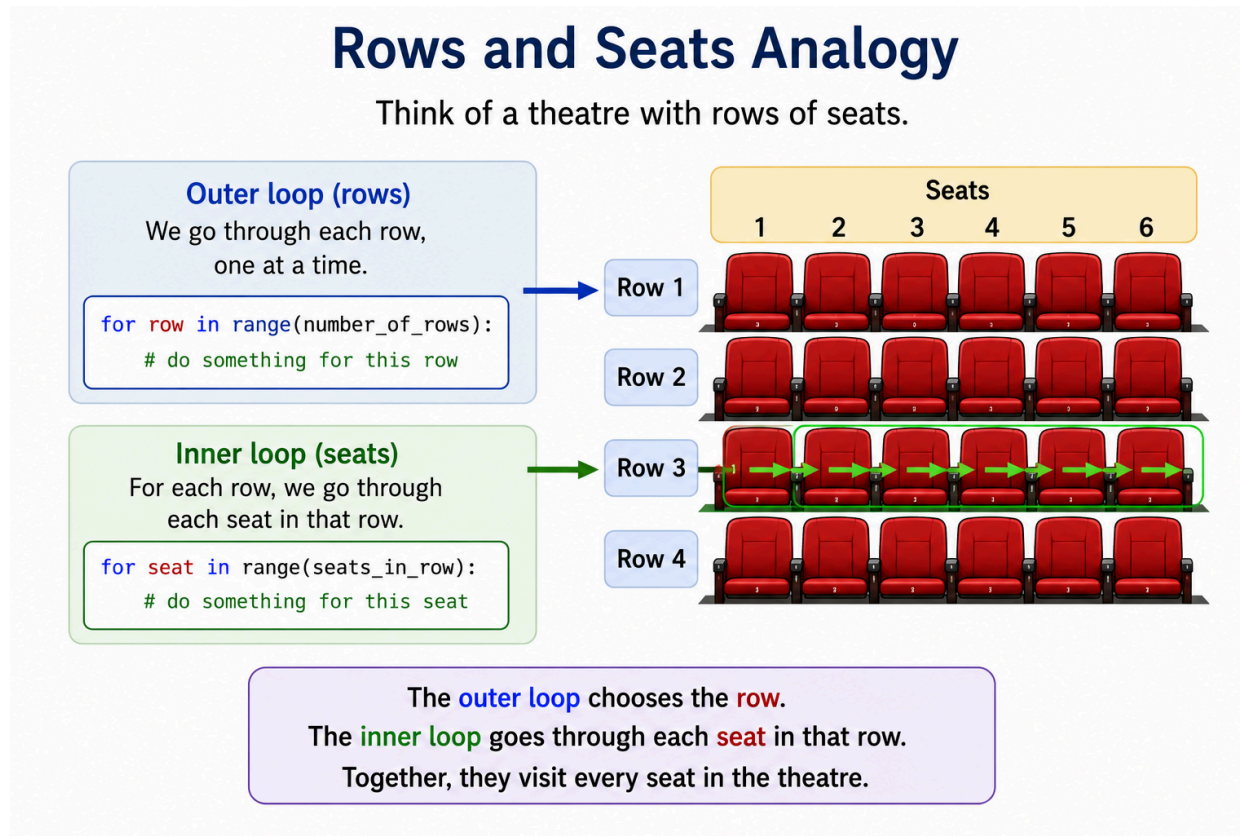
The **inner loop** selects each seat in that row.

Rows and Seats Analogy

Think of a theatre with rows of seats.



8.3 Rows and Seats Analogy Continued



8.4 Rows and Seats Analogy Code

```
for row in range(4):  
    for seat in range(6):  
        print(row, seat)
```

8.5 Trace the idea

```
row 0: seat 0, seat 1, seat 2, seat 3, seat 4, seat 5
row 1: seat 0, seat 1, seat 2, seat 3, seat 4, seat 5
row 2: seat 0, seat 1, seat 2, seat 3, seat 4, seat 5
row 3: seat 0, seat 1, seat 2, seat 3, seat 4, seat 6
```

The inner loop runs completely for each value of row.

8.6 Is this code better?

```
for row in range(1,5):  
    print("\nRow ", row, end=' ')  
    for seat in range(1,7):  
        print(" Seat ", seat, end=' ')
```

Outline

1. Learning outcomes

2. Why another loop?

3. For loop basics

4. Using range()

5. Accumulators with for loops

6. Iterating through strings

7. Iterating through lists

8. Nested loops

9. Pattern examples

10. Common mistakes

11. Bringing it together

12. Summary

9.1 Print a square

```
for row in range(5):  
    print("*" * 5)
```

Output:

```
*****  
*****  
*****  
*****  
*****
```

9.2 Print a triangle

```
for row in range(1, 6):  
    print("*" * row)
```

Output:

```
*  
**  
***  
****  
*****
```

9.3 Multiplication table

```
for i in range(1, 11):  
    print(i, "x 7 =", i * 7)
```

9.4 1 to 10 tables

```
for table in range(1, 11):  
    print("Table", table)  
  
    for i in range(1, 11):  
        print(i, "x", table, "=", i * table)  
  
    print()
```

Outline

1. Learning outcomes

2. Why another loop?

3. For loop basics

4. Using range()

5. Accumulators with for loops

6. Iterating through strings

7. Iterating through lists

8. Nested loops

9. Pattern examples

10. Common mistakes

11. Bringing it together

12. Summary

10.1 Off-by-one errors

```
for i in range(1, 10):  
    print(i)
```

This prints 1 to 9, not 1 to 10.

The stop value is excluded.

10.2 Indentation errors

```
for i in range(5):  
    print(i)
```

Python needs indentation to show what belongs inside the loop.

10.3 Changing the wrong variable

```
total = 0
```

```
for mark in [50, 60, 70]:  
    total = mark
```

This replaces the total each time.

Better:

```
total += mark
```

Outline

1. Learning outcomes

2. Why another loop?

3. For loop basics

4. Using range()

5. Accumulators with for loops

6. Iterating through strings

7. Iterating through lists

8. Nested loops

9. Pattern examples

10. Common mistakes

11. Bringing it together

12. Summary

11.1 Mini-program: mark report

```
marks = [72, 38, 55, 90, 41]
```

```
total = 0
```

```
passes = 0
```

```
highest = marks[0]
```

```
for mark in marks:  
    total += mark
```

```
    if mark >= 40:  
        passes += 1
```

```
    if mark > highest:  
        highest = mark
```

11.1 Mini-program: mark report

```
print("Average:", total / len(marks))  
print("Passes:", passes)  
print("Highest:", highest)
```

11.2 What does this combine?

- Lists
- for loops
- Selection
- Counters
- Accumulators
- Highest value pattern
- `len()` function

Outline

1. Learning outcomes

2. Why another loop?

3. For loop basics

4. Using range()

5. Accumulators with for loops

6. Iterating through strings

7. Iterating through lists

8. Nested loops

9. Pattern examples

10. Common mistakes

11. Bringing it together

12. Summary

12.1 Today we learned

- Use for loops to process a sequence.
- `range()` generates numbers.
- The stop value in `range()` is excluded.
- Strings and lists can be looped through directly.
- Selection can be placed inside loops.
- Nested loops are useful for grids, tables and patterns.

12.2 Practice exercises

1. Print numbers from 1 to 50.
2. Print multiples of 3 from 3 to 60.
3. Print each character in your name.
4. Count vowels in a sentence.
5. Calculate the total of a list of numbers.
6. Count how many marks are passes.
7. Find the highest number in a list.
8. Print a 5 by 5 square of stars.
9. Print a right-angled triangle.
10. Challenge: print multiplication tables from 1 to 10.

Thanks for Watching - Any questions?

